

Algorithmique et programmation impérative

Chapitre 5 : Récursivité

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 5. Récursivité

- 1 Récursivité
- 2 Un langage complet (parenthèse)
- 3 Quelques algorithmes récursifs
- 4 Paramètres par défaut et arguments mots-clefs (parenthèse)
- 5 Erreurs fréquentes et debug
- 6 Exercice interactif
- 7 Glossaire

Récursivité

Illustration de la récursivité

Illustration en auditoire : énigme présentée en auditoire et résolue par un script dont le code sera dévoilé plus tard.

Preuve par récurrence

La récursivité est une notion classique en mathématiques quand il s'agit de prouver une assertion (preuve par récurrence) : cf. cours *Mathématiques élémentaires*

Rappel du principe

A prouver : assertion $S(n) = f(n) \quad \forall n \geq n_0$

Base : preuve de l'assertion pour certaines petites valeurs de n (n_0 par exemple)

Etape de récurrence : on suppose que c'est vrai pour $n \leq k$ (avec $k \geq n_0$), prouver que c'est également vrai pour $n = k + 1$

Remarque : on peut aussi supposer que c'est vrai pour $n \leq k - 1$ et prouver que c'est vrai pour $n = k$. C'est ce qu'on fera ici : souvent plus intuitif d'un point de vue informatique.

Preuve par récurrence

Exemple

Soit $S(n) = \sum_{i=1}^n i$. Prouver que

$$S(n) = \frac{n(n+1)}{2} \quad \forall n \geq 1 \quad (1)$$

Base : si $n = 1$, alors $S(1) = 1$ par définition et (1) devient

$$1 = \frac{1 \times 2}{2},$$

ce qui est exact.

Preuve par récurrence

Etape de récurrence : on suppose que (1) est vraie pour $1 \leq n \leq k-1$, prouvons que c'est également vrai pour $n = k$. Par hypothèse de récurrence,

$$S(k-1) = \frac{(k-1)k}{2} \quad (2)$$

Comment exprimer $S(k)$ en fonction de $S(k-1)$? Par définition,

$$S(k) = \sum_{i=1}^k i = \sum_{i=1}^{k-1} i + k = S(k-1) + k.$$

Par (2), il s'ensuit

$$S(k) = \frac{(k-1)k}{2} + k.$$

Or,

$$\frac{(k-1)k}{2} + k = \frac{k^2 - k}{2} + \frac{2k}{2} = \frac{k^2 + k}{2} = \frac{k(k+1)}{2}.$$

Preuve par récurrence

On a donc bien

$$S(n) = \frac{n(n+1)}{2} \quad \forall n \geq 1.$$

Qu'avons-nous du déterminer pour utiliser une preuve par récurrence ?

- (base) résoudre un cas simple
- (étape de récurrence)
 - ▶ supposer le résultat vrai pour $n \leq k - 1$
 - ▶ exprimer $S(k)$ en fonction de $S(k - 1)$ (ou en fonction d'autres valeurs plus petites que $k - 1$)
 - ▶ retrouver le résultat en combinant les deux points ci-dessus

Récursivité

En informatique, on peut également utiliser la **récursivité**

Problème

Comment calculer la somme des n premiers nombres entiers ?

Solution 1 : utiliser la formule prouvée précédemment (formule d'Euler)

```
>>> def sum_1_to_n(n):  
    return n * (n + 1) / 2
```

```
>>> print(sum_1_to_n(10))
```

55

Récursivité

Solution 2 : utiliser la récursivité, c'est-à-dire la possibilité pour une fonction de s'appeler elle même.

- (base) Si $n = 1$, alors la somme $S(n)$ vaut 1
- (étape de récurrence) Si $n > 1$, $S(n) = S(n - 1) + n$

Ces deux éléments suffisent pour résoudre le problème de manière récursive

```
>>> def sum_rec(n):  
    if n == 1:  
        return 1  
    else:  
        return sum_rec(n - 1) + n
```

```
>>> print(sum_rec(10))
```

55

Récursivité

Intuition : le programmeur paresseux

Supposons que Bill soit paresseux et qu'il doive résoudre un problème complexe. Il préfère laisser à Bob le soin de faire le gros du travail. Si Bill est capable de

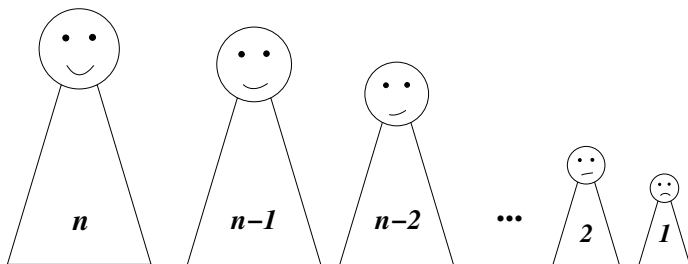
- résoudre le problème pour les cas les plus simples ;
- récupérer le travail de Bob et réaliser un travail de taille un peu plus grande à partir de celui-ci ;

alors le problème peut être résolu par récursivité, de manière simple.

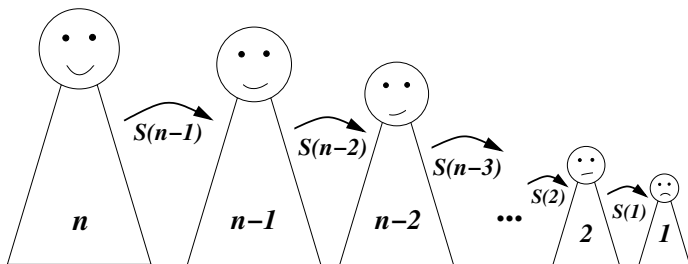
Exemple : Bill doit calculer $S(n) = \sum_{i=1}^n i$. Bill peut demander à Bob de calculer $S(k)$ pour $k = 2, 3, \dots, n-1$. Il lui reste simplement à

- calculer $S(1)$,
- déterminer comment calculer $S(n)$ en utilisant $S(n-1)$ (ou d'autres valeurs calculées par Bob).

Récursivité



Récursivité



Récursivité

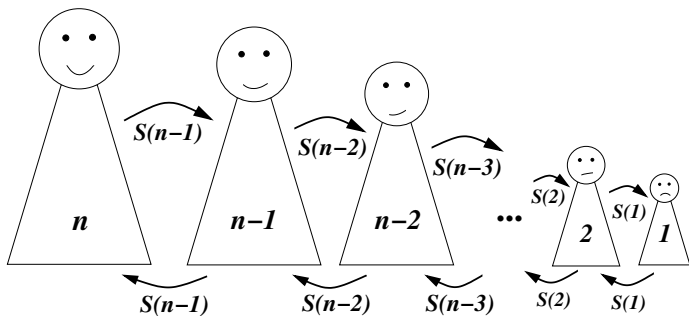


Illustration : appel à `sum_rec(4)` dans Python Tutor

Récursivité

Problème

Ecrire une fonction qui calcule la factorielle d'un nombre entier.

Récursivité

Problème

Ecrire une fonction qui calcule la factorielle d'un nombre entier.

Une solution récursive :

- (base) $0! = 1$
- (étape de récurrence) $n! = n \times (n - 1)!$

Récursivité

En Python :

```
>>> def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        recurse = factorial(n - 1)  
        result = n * recurse  
        return result  
  
>>> print(factorial(4))
```

24

Récursivité

Problème

Ecrire une fonction qui calcule a^n , si $n \geq 1$.

Récursivité

Problème

Ecrire une fonction qui calcule a^n , si $n \geq 1$.

Une solution récursive :

- (base) $a^1 = a$
- (étape de récurrence) $a^n = a^{n-1} \times a$

Récursivité

En Python :

```
>>> def expo(a, n):  
    if n == 1:  
        return a  
    else:  
        return expo(a,n - 1) * a
```

```
>>> print(expo(3,3), (3**3))
```

27 27

Rappels concernant les fonctions

Au chapitre 3, nous avons vu que :

- une fonction peut être utilisée dans une expression comme une valeur (= sa valeur de retour)
- une fonction peut en appeler une autre

La récursivité n'est rien d'autre que l'application de ses deux faits sur la fonction elle-même !

```
def expo(a, n):  
    if n == 1:  
        return a  
    else:  
        return expo(a, n - 1) * a
```

Pour comprendre et valider une fonction récursive, il s'agit

- d'accepter qu'un appel récursif retourne la bonne valeur
- de s'assurer que le(s) cas de base(s) soi(en)t bien géré(s)
- de s'assurer que chaque appel récursif réduise la taille du problème jusqu'à atteindre un cas de base

Récursivité

Les exemples précédents montrent que l'on peut résoudre très simplement certains problèmes si on arrive à :

- résoudre le problème pour les cas les plus simples ;
- récupérer une solution pour une certaine taille du problème et construire une solution de plus grande taille à partir de celui-ci.

Dans beaucoup de cas, ces deux tâches sont plus faciles à résoudre qu'une solution "générale" ou "complète" au problème !

Mais... cela peut paraître "magique" : on peut avoir du mal à "accepter" que cela fonctionne aussi simplement.

On peut analyser le flot d'exécution pour s'en convaincre.

Récursivité et flot d'exécution

```
>>> def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        recurse = factorial(n - 1)  
        result = n * recurse  
        return result
```

```
>>> print(factorial(3))
```

6

⇒ analyse du flot d'exécution via Python Tutor

Récursivité et flot d'exécution

Problème

Soit la fonction récursive suivante, comment prédire ce qu'elle va faire ? Essayez de décrire le flot d'exécution après un appel à `post_count(3)`.

```
>>> def post_count(n):  
    if n <= 0:  
        print('Boum')  
    else:  
        print(n)  
        post_count(n - 1)  
>>> post_count(3)
```

Analyse du flot d'exécution lors de l'appel `post_count(3)`

```
>>> post_count(3)
3
2
1
Boum
```

⇒ analyse du flot d'exécution via Python Tutor

Récursivité et flot d'exécution

Problème

Que se passe-t-il si on modifie légèrement la fonction précédente en déplaçant l'appel récursif **avant** l'instruction “print(n)” ?

```
>>> def pre_count(n):  
    if n <= 0:  
        print('Boum')  
    else:  
        pre_count(n - 1)  
        print(n)  
>>> pre_count(3)
```

Récursivité et flot d'exécution

Pour résoudre ce problème, on peut refaire l'analyse du flot détaillée, mais ce n'est pas nécessaire si l'on “accepte” que la récursivité fonctionne et qu'on lit le corps du `else` comme suit :

Etape de recurrence:

```
pre_count(n - 1)
print(n)
```

Ce qui s'interprète par

- on réalise d'abord le travail pour les valeurs plus petites que `n`
- ensuite on affiche `n`

```
>>> pre_count(3)
```

Boum

```
1
2
3
```

Définitions récursives

La récursivité est très naturelle pour résoudre des problèmes qui sont définis récursivement.

Par exemple, la séquence des nombres de Fibonacci est définie mathématiquement de manière récursive :

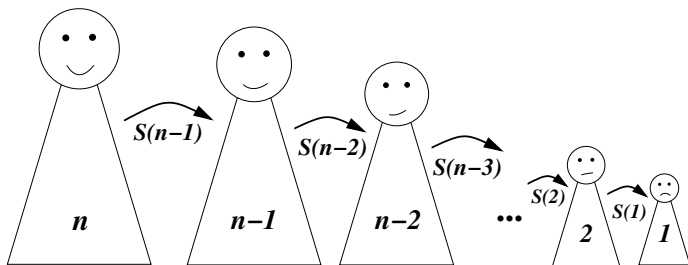
$$\begin{aligned}F_0 &= 0, \\F_1 &= 1, \\F_n &= F_{n-1} + F_{n-2}, \quad \forall n \geq 2.\end{aligned}$$

Définitions récursives

En Python :

```
def fibo(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibo(n - 1) + fibo(n - 2)
```

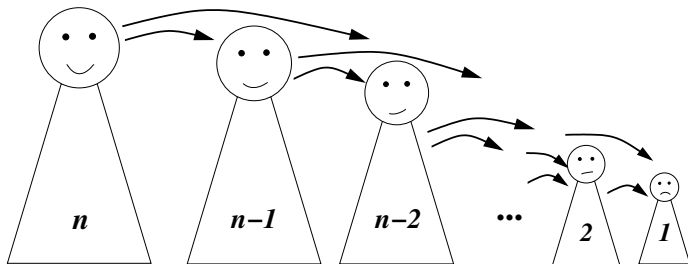
Récurrances faible et forte



Récurrance faible : n'utilise que la valeur précédente ($n - 1$).

Exemples : factorielle, exposant, etc. vus ci-avant.

Récurrances faible et forte



Récurrance forte : utilise plusieurs (voire toutes les) valeurs précédentes ($\leq n-1$).

Exemple : Fibonacci qui utilise les valeurs $F(n-1)$ et $F(n-2)$.

Récursion infinie

Si une récursion n'atteint jamais un cas de base, le programme ne se termine (théoriquement) jamais ! C'est une **récursion infinie**.

```
def recurse():  
    recurse()
```

→ voir la console interactive pour le comportement de cette fonction

Important de considérer les cas de base **et** le fait que chaque appel récursif doit s'approcher d'un cas de base.

Récursion infinie

En réalité, la récursion infinie n'existe (heureusement) pas, car une exception est lancée quand le nombre d'appels récursif dépasse une certaine limite. On peut la connaître (et la modifier) grâce à certaines fonctions du module `sys`.

```
>>> import sys
>>> print(sys.getrecursionlimit())
1000
>>> sys.setrecursionlimit(10000)
```

Si cette limite n'était pas définie, le programme “planterait” une fois que la mémoire allouée sur la machine pour appeler les milliers d'appels récursifs serait dépassée, ce qui serait bien plus gênant qu'une exception (gérable, voir plus tard).

Récursion infinie

Que se passe-t-il si on appelle `factorial` avec 1.5 comme argument ?

```
>>> sys.setrecursionlimit(10000)
```

```
>>> factorial(1.5)
```

```
...
```

```
RuntimeError: maximum recursion depth exceeded
```

La base ($n == 0$) n'est jamais atteinte :

$1.5 \rightarrow 0.5 \rightarrow -0.5 \rightarrow -1.5 \rightarrow \dots$

Tester les types

Pour éviter ce genre de problèmes, on peut utiliser la fonction booléenne `isinstance` qui vérifie si un argument est d'un type donné.

```
>>> def factorial(n):  
    if not isinstance(n, int):  
        print('Factorial is only defined for integers.')  
        return None  
    elif n < 0:  
        print('Factorial is only defined for positive integers.')  
        return None  
    elif n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

```
>>> factorial('fred')  
Factorial is only defined for integers.  
None  
>>> factorial(-2)  
Factorial is only defined for positive integers.  
None
```

Un langage complet (parenthèse)

Un langage complet

Jusqu'à présent nous n'avons couvert qu'une petite partie de Python, mais il s'agit pourtant déjà d'un **langage complet**, c.-à-d. que **tout** problème qui peut être résolu de manière algorithmique peut être exprimé dans ce langage !

- tout programme déjà écrit pourrait être réécrit en utilisant uniquement les caractéristiques du langage vues*
- le reste du langage permet de faire les choses plus facilement
- prouver cette affirmation n'est pas facile, mais cela a été fait par Alan Turing (mathématicien, 1912 – 1954, souvent considéré comme le père de l'informatique et qui a donné son nom à une de vos salles machines)



* En réalité, vous auriez besoin de quelques commandes supplémentaires pour contrôler des éléments comme la souris, les fichiers, etc.

Quelques algorithmes récursifs

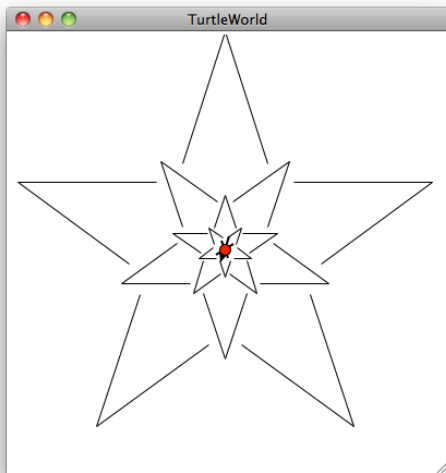
Quelques algorithmes récursifs

Nous allons illustrer / utiliser la récursivité

- en étudiant le comportement récursif de notre tortue (voir tutoriel “UTurtle”);
- en concevant un algorithme récursif pour calculer (une approximation de) la racine carrée d'un nombre ;
- en dévoilant le code qui a permis de résoudre l'énigme.

Une tortue récur­sive

Réalisation en auditoire du Tutoriel “UTurtle” et illustration de la récur­sivité.



La méthode de Héron

Pour approximer une racine carrée, on utilisera la méthode de Héron qui peut se résumer en une phrase :

Si x est une approximation de $\sqrt{a}$, alors $\frac{x+a/x}{2}$ est une meilleure approximation de $\sqrt{a}$.

Exercice

Ecrire une fonction qui calcule une approximation de la racine carrée d'un nombre réel.

Etape 1 : quelques essais dans la console interactive

Si x est une approximation de $\sqrt{a}$, alors $\frac{x+a/x}{2}$ est une meilleure approximation de $\sqrt{a}$.

Dans la console interactive : test “à la main” de la méthode de Héron pour calculer $\sqrt{4}$.

Première approximation utilisée : 1

```
>>> a = 4
>>> x = 1
>>> (x + a / x) / 2.0
2.5
>>> x = 2.5
>>> (x + a / x) / 2.0
2.05
>>> x = 2.05
>>> (x + a / x) / 2.0
2.000609756097561
```

Etape 2 : spécification de la fonction `racine`

On va écrire une fonction `racine` récursive pour appliquer la méthode de Héron.

Entrée :

Sortie :

Etape 2 : spécification de la fonction `racine`

On va écrire une fonction `racine` récursive pour appliquer la méthode de Héron.

Entrée : deux nombres réels a et x

Sortie : un nombre réel qui est une meilleure approximation de $\sqrt{a}$ que x

```
def racine(a, x):  
    pass # TODO
```

Etape 3 : cas de base

Le **cas de base** de la fonction `racine` équivaut au fait que l'approximation n'est plus significativement meilleure (car la méthode converge)

Etape 3 : cas de base

Le **cas de base** de la fonction `racine` équivaut au fait que l'approximation n'est plus significativement meilleure (car la méthode converge)

→ on s'arrête quand la nouvelle approximation est "égale" à la précédente (i.e., suffisamment proche, cf. chap. 4)

Etape 3 : cas de base

Le **cas de base** de la fonction `racine` équivaut au fait que l'approximation n'est plus significativement meilleure (car la méthode converge)

→ on s'arrête quand la nouvelle approximation est “égale” à la précédente (i.e., suffisamment proche, cf. chap. 4)

Il ne faut pas oublier de retourner cette approximation finale pour permettre que cette information “remonte”.

Interlude : tester l'égalité de 2 réels

Comme expliqué au Chap. 4, on ne teste pas si 2 valeurs réelles sont égales mais on vérifie qu'elles soient suffisamment proches.

```
>>> import math
>>> def est_egal(x, y):
    eps = 10**-9
    return abs(x - y) < eps

>>> x = 0.0
>>> y = math.cos(math.pi / 2.0)
>>> x == y
False
>>> est_egal(x, y)
True
>>> print(x, y)
0.0 6.123233995736766e-17
```

Etape 3 : cas de base

```
def racine(a,x):  
    new = (x + a / x) / 2.0  
    if est_egal(x, new):  
        return x  
    else:  
        pass # cas recursif TODO
```

Etape 4 : cas récursif

Si x est une approximation de $\sqrt{a}$, alors $\frac{x+a/x}{2}$ est une meilleure approximation de $\sqrt{a}$.

Le **cas récursif** devient facile : il sert à faire “remonter” l’approximation finale en appliquant littéralement la méthode de Héron.

```
def racine(a, x):
    new = (x + a / x) / 2.0
    if est_egal(x, new):
        return new
    else:
        return racine(a, new)
```

⇒ démonstration avec **Python Tutor** quand on oublie le `return`

Etape 5 : fonction “wrapper”

La fonction récur­sive `racine` nécessite deux paramètres en entrée, alors que seul a est nécessaire pour calculer $\sqrt{a}$ d'un point de vue “utilisateur”.

On peut “emballer” (to wrap) le premier appel récur­sif dans une fonction qui initialisera les paramètres supplémentaires (ici, la première approximation).

```
>>> def racine_carree(a):  
    return racine(a, 1)  
  
>>> racine_carree(4)  
2.0  
>>> est_egal(racine_carree(4), 2)  
True
```

Une autre solution : utiliser un paramètre par défaut (voir parenthèse ci-après).

Résolution de l'énigme

Illustration : présentation du code de l'énigme résolue en début de séance.

Paramètres par défaut et arguments mots-clefs (parenthèse)

Paramètres par défaut

Un **paramètre par défaut** est une valeur par défaut que l'on assigne à un paramètre lors de la définition d'une fonction. Cela permet de rendre un paramètre optionnel.

```
>>> def racine(n, x = 1):  
    ...
```

```
>>> racine(4)  
2.0
```

Les paramètres optionnels doivent se trouver après les paramètres obligatoires. Avec les fonctions récursives, ils sont utiles pour cacher des paramètres auxiliaires sans devoir écrire une fonction “wrapper”.

Arguments “mots-clefs”

Un **argument mot-clef** est un argument nommé lors d'un appel à une fonction. Cela permet de modifier l'ordre des arguments, ce qui peut être utile avec des arguments optionnels.

```
>>> def f(x, y = 2, z = 3):
    print('x = ' + str(x) + ', y = ' + str(y) + ', z = ' + str(z))

>>> f(6, 5, 7)
x = 6, y = 5, z = 7
>>> f(7)
x = 7, y = 2, z = 3
>>> f(9, 10)
x = 9, y = 10, z = 3
>>> f(7, z = 8)
x = 7, y = 2, z = 8
>>> f(y = 2, z = 1)
TypeError: f() takes at least 1 non-keyword argument (0 given)
```

Arguments “mots-clefs”

Vous pouvez imposer que certains arguments soient obligatoirement **positionnés** et ne peuvent dès lors pas être utilisés comme arguments mots-clefs en les plaçant devant un « , / »

```
>>> def f(x, y = 2, /, z = 3):  
    print('x = ' + str(x) + ', y = ' + str(y) + ', z = ' + str(z))  
  
>>> f(7, z = 8)  
x = 7, y = 2, z = 8  
>>> f(7, y = 8)  
TypeError: f() got some positional-only arguments passed as keyword arguments: 'y'
```

Remarque : c'est ce que nous avons parfois vu précédemment dans l'aide de certaines fonctions.

Erreurs fréquentes et debug

Erreurs fréquentes et debug

Pour les erreurs de syntaxe et les exceptions, Python affiche un message : l'information la plus utile dans ce message est :

- le type d'erreur (dernière ligne) : `SyntaxError`, `RuntimeError`, ...
- l'endroit où cette erreur apparaît.

Cependant, il n'est pas toujours facile de trouver l'endroit *exact* du code qu'il faut corriger avec ces informations.

Erreurs fréquentes et debug

Erreur d'indentation :

```
>>> x = 5
>>> y = 6
File "<stdin>", line 1
    y = 6
    ^
IndentationError: unexpected indent
```

Erreurs fréquentes et debug

Exceptions sur les valeurs dans des opérations mathématiques (ici, dues à la division entière qui n'aurait pas du être utilisée)

```
>>> import math
>>> signal = 9
>>> noise = 10
>>> ratio = signal // noise
>>> decibels = 10 * math.log10(ratio)
File "<stdin>", line 1, in <module>
    decibels = 10 * math.log10(ratio)
ValueError: math domain error
>>> invert = 1 / ratio
File "<stdin>", line 1, in <module>
    invert = 1 / ratio
ZeroDivisionError: integer division by zero
```

Ici : le message n'indique pas où se trouve le problème (qui devrait être corrigé lors du calcul de `ratio`)

Erreurs fréquentes et debug

Si on obtient une exception ou une erreur sémantique lors de l'appel d'une fonction, il y a 3 possibilités :

- problème au niveau des arguments passés à la fonction : une **précondition**¹ est violée (par exemple un argument dont le type n'est pas géré par la fonction, ou une valeur négative pour un calcul sur des nombres positifs)
- problème au niveau du code de la fonction (corps) : une **postcondition**² est violée (par exemple, une erreur sémantique dans le corps de la fonction qui produit un mauvais résultat)
- problème au niveau de la valeur de retour (est-elle correctement utilisée ?)

1. conditions que la fonction suppose vraies pour les entrées, avant d'exécuter son travail

2. conditions qu'une fonction garantit si les préconditions sont respectées, la validité de son résultat

Erreurs fréquentes et debug

Déboguer une fonction “à la main” :

- 1 Vérifier le respect des préconditions : afficher la valeurs des paramètres en début de fonction (et peut-être leur types) et / ou écrire du code qui vérifie les préconditions explicitement.
- 2 Si les paramètres semblent corrects, afficher les valeurs retournées avant chaque instruction `return` et vérifier les résultats à la main.
- 3 Si la fonction semble correcte, regardez l'appel de la fonction et vérifiez que la valeur de retour est utilisée correctement (ou qu'elle est vraiment utilisée).
- 4 Afficher certains messages au début et à la fin d'une fonction peut aider à rendre plus visible le flot d'exécution.

Remarque : plus tard, vous utiliserez des **débogueurs** qui permettent de “tracer” un programme pas à pas et de consulter les valeurs des variables à chaque étape. Vous pouvez également utiliser Python Tutor pour des petits morceaux de code.

Exercice interactif

Exercice

Dans le code suivant,

- Où est le cas de base de la fonction récursive `mystery` et comment y arrive-t-on ?
- Que va dessiner la tortue ?

```
from uturtle import *  
  
def mystery(t, x, y):  
    moveForward(t, x)  
    turnRight(t, y)  
    if x > 0:  
        mystery(t, x - 1, y)  
  
bob = umonsTurtle()  
mystery(bob, 50, 15)
```

Glossaire

Glossaire

récursivité : le fait pour une fonction de s'appeler elle-même

cas de base : branche conditionnelle dans une fonction récursive qui ne fait pas d'appel récursif

réursion infinie : fonction qui s'appelle elle-même indéfiniment (sans atteindre jamais un cas de base). Provoque une exception.

paramètre par défaut : valeur par défaut que l'on assigne à un paramètre et qui le rend optionel.

argument mot-clef : argument nommé lors d'un appel à une fonction. Cela permet de modifier l'ordre des arguments passés lors de l'appel.